

Since bkhack is deployed to the browser, an algebra was a much-needed abstraction for parallelization using Workers. We introduced **fiber** as a library that implements structured concurrency—a concurrency algebra for internal bkhack modules. It is an extension of JavaScript Promise equipped with *control capabilities* i.e. to cancel and perform group actions.

For its design, we want the programming interface to be not only lean—where it should wraps over Worker as straightforwardly as possible—but also expressive—where programming patterns can be achieved as straightforwardly logical for the human programmer as possible. Our fiber heavily uses the concept of object algebra [Ext] in the form of a *ctrl* (pronounced “controller”) to both manage fibers transparently and provides a dictionary of encoder-decoder functions at call-site. Assume we’re writing a small program to compute self-exponential with interactions with a reactive-programming system. Given a staged code k defined by

$$\begin{array}{l} \text{bind : offshore to Worker}^1 \\ \quad \mid \\ \text{let}_{\text{Fiber.bind}} k = (f, n) \Rightarrow \\ \quad \left\{ \begin{array}{l} \text{let}_{\star} (\cdot) = \text{Lam.app}(n^n - 2n + 1, f) \\ \text{return } n \end{array} \right. \end{array}$$

it’s possible to run k as a background thread in the form of a continuation. So,

```
let ctrl = Fiber.create(·)
and f = i ⇒ return setCount(_ ⇒ i)
let★ result =
  Promise.all Iter.range(1000, n ⇒
    Fiber.With-ctrl.run-promise2(~ctrl,
      ctrl→Fiber.lam(f), n, k))
```

. Notice that *ctrl*’s algebra contains at least a lambda-like lambda which was used to wrap the effect-ful callback f , and a zoo of other terms.

Design

The *module layer*.

The *ppx layer*.

The *code extraction & generation layer*

main	hall
------	------

Implementation

Workers have specific characteristics that makes its programming low-level [MS]. First, Workers can’t send messages directly to each other. Secondly, Workers can’t reply to messages, can’t correlate messages sent to, and later received from, the main workers. Third, Workers can’t block, can’t pause processing of the current message in anticipation of another message.

Related works

Unlike most solutions in the ecosystem, fiber offers a more opaque interface. Dune’s fiber. The Cancel module. Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do. Eio’s Fiber. Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do.

Promise-based asynchronous task-running [Zak16] Reo@JS is a JavaScript runtime that enables high-level Reo-based worker coordination [MS], an attempt to abstract in-browser parallelism similar to us. Reo@JS emphasizes the importance of coordination protocols as explicit programming artifacts, it uses the Reo circuit-based coordination language then compiles it to JavaScript. Its API is a mix of Promise-based and generators. Our implemetation is Promise-based in a functional programming language.

[Ext] “Extensibility for the Masses: Practical Extensibility with Object Algebras,” in *The European Conference on Object-Oriented Programming, ECOOP 2012*,

[MS] Marco Krauweel and Sung-Shik T.Q. Jongmans, “Simpler Coordination of JavaScript Web Workers.”

[Zak16] N. C. Zakas, “Promises and Asynchronous Programming,” in *Understanding ECMAScript 6*, 1st edition., No Starch Press, 2016, ch. 11, pp. 213–241.

¹for more details on what is *offshoring*, refers to X